MODULE 24 · DOMAIN 3 - CLOUD NATIVE APPLICATION DELIVERY

GitOps

The four principles, Argo CD and ApplicationSets, Flux, drift, and the mistakes to avoid.

DOMAIN 3 of 4 EXAM WEIGHT 16% MODULE 24 of 35

admin@sthithapragna.com

In GitOps, **Git is the source of truth**, and **an agent in the cluster** keeps reality matching it.

The four OpenGitOps principles

PRINCIPLE	MEANS
Declarative	The desired state is expressed declaratively, not as steps
Versioned and immutable	Stored so that every version is kept, with complete history
Pulled automatically	Agents pull the desired state from the source
Continuously reconciled	Agents keep observing, and correct any difference

ON THE EXAM

Defined by the OpenGitOps project, part of the CNCF. Pushed by a pipeline is not one of them.

The GitOps loop

01
Propose
A change is made in Git, as a pull request.

02
Review
Colleagues review and approve; it is merged.

03
Notice
The agent sees the new commit.

04
Compare
Git's desired state versus the cluster's actual state.

05
Apply
Differences are applied; the agent keeps watching.

ON THE EXAM

Every change to production is a reviewed, recorded commit.

Why teams adopt GitOps

- **Audit:** git log shows who changed what, when, and who approved it.
- **Rollback:** revert the commit, and the cluster follows.
- **Security:** nothing outside the cluster holds cluster credentials.
- **Drift correction:** manual changes are detected, and can be reverted.
- **Recovery:** a lost cluster can be rebuilt from the repository.

> ON THE EXAM
>
> Most GitOps benefits come from one fact: the cluster's desired state is a Git repository.

Argo CD

- **A graduated CNCF project,** part of the Argo family.
- **An Application** links a source in Git to a destination cluster and namespace.
- **Understands plain YAML, Kustomize and Helm** as sources.
- **A web UI and CLI** show every application's sync and health status.

> ON THE EXAM
>
> Argo CD's core object is the Application: what to deploy, from where, to where.

An Argo CD Application for the booking API

```
 1 apiVersion: argoproj.io/v1alpha1
 2 kind: Application
 3 metadata: {name: booking-api-prod, namespace: argocd}
 4 spec:
 5   project: default
 6   source:
 7     repoURL: https://git.quayside.example/platform/deploy.git
 8     targetRevision: main
 9     path: booking-api/overlays/prod
10   destination: {server: https://kubernetes.default.svc, namespace: quayside}
11   syncPolicy:
12     automated: {prune: true, selfHeal: true}
```

9 Module 23's Kustomize overlay, straight from Git.

10 Where to deploy: this cluster, the quayside namespace.

12 Sync automatically, delete what Git removes, revert drift.

Sync status versus health status

STATUS	VALUES	ANSWERS
Sync	Synced, OutOfSync	Does the cluster match Git?
Health	Healthy, Progressing, Degraded, Missing	Is the application actually working?

ON THE EXAM

Synced but Degraded: the cluster matches Git, and what Git describes is broken. Fix it in Git.

prune versus selfHeal

PRUNE

Deletes what Git removed

- A manifest is deleted from Git
- The matching object is deleted in the cluster
- Without it: the object lingers
- Stops leftovers building up

SELFHEAL

Reverts manual changes

- Someone edits an object in the cluster
- Argo CD changes it back to match Git
- Without it: drift stays until the next commit
- Makes Git the only way to change things

THE TIEBREAKER

Both are off by default, even with automated sync. Turn them on deliberately.

ApplicationSets: Applications from a template

- **An ApplicationSet** is a template plus generators that produce Applications.
- **List generator:** a fixed list, such as three environments.
- **Cluster generator:** one Application per registered cluster.
- **Git generator:** one Application per folder or file in a repository.
- **Matrix and merge** combine generators; a pull request generator builds preview environments.

> ON THE EXAM
>
> 'An Application for every folder, including ones added later': an ApplicationSet with a Git generator.

One Application per service, automatically

```yaml
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata: {name: quayside-prod, namespace: argocd}
spec:
  generators:
  - git:
      repoURL: https://git.quayside.example/platform/deploy.git
      revision: main
      directories: [{path: "*/overlays/prod"}]
  template:
    metadata: {name: "{{path[0]}}-prod"}
    # spec: source path "{{path}}", destination as before
```

9 Every folder matching this pattern becomes an Application.

11 Named from the folder: booking-api-prod, web-prod, and so on.

THE TAKEAWAY

Add a new service's folder in Git, and its Application appears. Delete it, and the Application goes.

Flux

- **A graduated CNCF project,** built as a set of controllers: the GitOps Toolkit.
- **source-controller** fetches Git repositories, OCI artefacts and Helm repositories.
- **kustomize-controller** applies manifests, through a Flux Kustomization object.
- **helm-controller** manages HelmRelease objects; others handle notifications and image updates.

> ON THE EXAM
>
> A Flux Kustomization is a custom resource, not the kustomization.yaml file. The names catch people out.

Argo CD versus Flux

ARGO CD

Application-centred, with a UI

- Application and ApplicationSet objects
- Rich web UI for many clusters
- Central instance can manage many clusters
- Plain YAML, Kustomize, Helm

FLUX

A toolkit of controllers

- GitRepository, Kustomization, HelmRelease
- CLI-first; UIs come from elsewhere
- Often runs inside each cluster
- Built-in image update automation

THE TIEBREAKER

Both are graduated, pull-based GitOps. Pick by the detail: UI or ApplicationSet means Argo CD; HelmRelease or toolkit means Flux.

Promotion and rollback, the GitOps way

- **Promote** with a pull request that changes the image tag in the production overlay.
- **Roll back** with git revert. The agent syncs the previous state.
- **Not with kubectl rollout undo:** self-heal puts the bad version straight back.
- **Automate tag bumps** with Argo CD Image Updater or Flux image automation.

> ON THE EXAM
>
> In GitOps, every change goes through Git, including emergencies and rollbacks.

Secrets in a GitOps repository

- **Never commit plain Secrets.** Base64 is not encryption, as module 9 showed.
- **Sealed Secrets:** encrypt for the cluster; only its controller can decrypt.
- **SOPS:** encrypted files in Git, decrypted by Flux or a plugin at apply time.
- **External Secrets:** Git holds a reference; values come from Vault or a cloud secret manager.

> ON THE EXAM
>
> Git holds everything except secret values, which are encrypted or fetched from a secret store.

GitOps anti-patterns

- **kubectl edit in production:** reverted by self-heal, or left as silent drift.
- **Two writers:** a CI pipeline also running kubectl apply against the same cluster.
- **A long-lived branch per environment:** merges get painful and branches drift apart.
- **Prefer one branch, a folder or overlay per environment,** promoted by pull request.

> ON THE EXAM
>
> One source of truth, one writer. Anything else is not really GitOps.

Six GitOps decisions. For each, ask: what does Git say, and what will the agent do about it?

> Quayside's platform team is writing an internal guide to GitOps, listing the four OpenGitOps principles.
>
> A reviewer spots that one item on the draft list is not a principle.

Which item is not an OpenGitOps principle?

A Desired state is expressed declaratively

B Desired state is pulled automatically

C The system is continuously reconciled

D Changes are pushed by the CI pipeline

◆ PAUSE AND DECIDE

Which item is not an OpenGitOps principle?

A Desired state is expressed declaratively

Principle 1: declarative.

B Desired state is pulled automatically

Principle 3: pulled automatically.

C The system is continuously reconciled

Principle 4: continuously reconciled.

D Changes are pushed by the CI pipeline

The opposite of GitOps: agents pull; pipelines do not push.

THE SIGNAL

Declarative, versioned and immutable, pulled, reconciled. Pushed is the odd one out.

> During an incident, an engineer runs kubectl scale to raise the booking API from 3 to 10 replicas in production.
>
> Within a minute it is back to 3. The Argo CD Application uses automated sync with selfHeal: true. No autoscaler exists.

Why did the replica count return to 3?

A An autoscaler scaled it back down

B Argo CD self-healed the drift from Git

C The Deployment controller reverted it

D etcd restored an earlier snapshot

◆ PAUSE AND DECIDE

Why did the replica count return to 3?

A An autoscaler scaled it back down

The scenario says no autoscaler exists.

B Argo CD self-healed the drift from Git

Git says 3; self-heal reverts the manual change.

C The Deployment controller reverted it

It makes Pods match the spec; it never edits the spec.

D etcd restored an earlier snapshot

etcd never restores state on its own.

THE SIGNAL

Manual change reverted within a minute, selfHeal on: drift correction. Change Git instead.

> The team deletes the old nightly-report CronJob's manifest from Git. Argo CD applies other changes automatically.
>
> But the CronJob still runs every night, and Argo CD marks it as needing to be pruned.

Which setting makes Argo CD delete such objects automatically?

A prune: true in the automated sync policy

B selfHeal: true in the automated sync policy

C targetRevision: HEAD instead of main

D A Replace=true sync option

◆ PAUSE AND DECIDE

Which setting makes Argo CD delete such objects automatically?

A prune: true in the automated sync policy

Deletes objects whose manifests were removed from Git.

B selfHeal: true in the automated sync policy

Reverts edits to existing objects; it does not delete removed ones.

C targetRevision: HEAD instead of main

Changes which commit is tracked, not deletion.

D A Replace=true sync option

Changes how objects are applied, not whether they are deleted.

THE SIGNAL

Removed from Git but still in the cluster: turn on prune.

> Quayside has thirty microservices, each with a production overlay folder in the deployment repository.
>
> It wants one Argo CD Application per service, created automatically, including for services added next year.

What should the team use?

A Thirty Applications, written and maintained by hand

B One Application pointing at the repository root

C A CronJob that runs kubectl apply on every folder

D An ApplicationSet with a Git directory generator

◆ PAUSE AND DECIDE

What should the team use?

A Thirty Applications, written and maintained by hand

Works today, but every new service needs a new file.

B One Application pointing at the repository root

One giant app: no per-service health, sync or rollback.

C A CronJob that runs kubectl apply on every folder

Push-style, no reconciliation: not GitOps.

D An ApplicationSet with a Git directory generator

A folder appears in Git; its Application appears too.

THE SIGNAL

'One Application per folder, including future ones': ApplicationSet, Git generator.

> A bad release of the web front end reached production through Argo CD, with selfHeal enabled.
>
> The team wants to return to the previous version quickly, in a way that sticks.

What should they do?

A. Run kubectl rollout undo on the web Deployment

B. Delete the Argo CD Application for web

C. Revert the release commit in Git

D. Edit the image tag directly in the cluster

◆ PAUSE AND DECIDE

What should they do?

A Run kubectl rollout undo on the web Deployment

Self-heal sees drift and puts the bad version back.

B Delete the Argo CD Application for web

Stops management, and may delete the running app.

C Revert the release commit in Git

Git goes back; the agent syncs the old version.

D Edit the image tag directly in the cluster

Drift again: reverted within a minute.

THE SIGNAL

GitOps rollback: git revert. Anything done only in the cluster is undone.

> The Argo CD dashboard shows the booking API Application as Synced, but Degraded.
>
> The on-call engineer asks what that combination means.

What does it mean?

A The cluster and Git differ, and Argo CD will fix it

B The cluster matches Git, but the app is unhealthy

C Argo CD cannot reach the Git repository

D Some resources defined in Git are missing

◆ PAUSE AND DECIDE

What does it mean?

A The cluster and Git differ, and Argo CD will fix it

That would be OutOfSync.

B The cluster matches Git, but the app is unhealthy

Git was applied faithfully; what Git describes is broken.

C Argo CD cannot reach the Git repository

That shows as a sync error, not Degraded.

D Some resources defined in Git are missing

That shows as health Missing, or OutOfSync.

THE SIGNAL

Synced plus Degraded: the problem is in what Git says. Fix it in Git.

Module 24 in five lines

- OpenGitOps: **declarative, versioned and immutable, pulled automatically, continuously reconciled**.
- **Argo CD:** Applications, sync versus health, prune and selfHeal (both off by default).
- **ApplicationSets** generate Applications: list, cluster, Git, matrix, pull request.
- **Flux:** GitOps Toolkit controllers, Kustomization and HelmRelease objects.
- **Roll back with git revert;** encrypt or externalise secrets; one writer only.

GitOps deploys it. Next: **debugging applications** when what you deployed misbehaves.